

İçindekiler

proje-kiti — Kit Ne Yapıyor

1. Kit nedir, ne değildir
2. Dış bağımlılıklar — ne kullanıyor, ne için
3. Akış — /yeni-proje sekiz adımda ne yapar
4. On dokuz standart
5. Ajan kapıları — CLAUDE.md
6. Bu kit nasıl gelişiyor

proje-kiti — Kit Ne Yapıyor

Sürüm: 1.74.0 · Tarih: 2026-09-03 Depo: github.com/bariskose9/bariskose-skills

Bu belge, kitin **kurulumdan canlıya çıkışa kadar** ne yaptığını anlatır. Terimlerin İngilizcesi yanlarında parantez içinde verilmiştir.

1. Kit nedir, ne değildir

Kit nedir: boş bir klasörde `/yeni-proje` yazıldığında, soruları cevaplayarak kurulmuş · test edilmiş · canlıya çıkmış bir proje elde etmeni sağlayan bir **Claude Code eklentisi** (plugin).

Kit ne değildir: özellik yazan bir araç değildir. Kurulumu bitirir ve yol haritasını (roadmap) çıkarır. Sonrasında proje adım adım ilerler ve **her adımda plan sunulup onay beklenir**.

Vaat "*tek promptla uygulama*" değil, "**tek promptla doğru kurulmuş proje ve net yol haritası**".

Dört komut

Komut	Ne yapar
<code>/yeni-proje</code>	Sıfırdan proje kurar, canlıya çıkarır
<code>/kit-senkron</code>	Projede öğrenilen kuralı kite geri yazar
<code>/video-analiz</code>	YouTube videosundan eksik best practice çıkarır
<code>/pdf-uret</code>	Markdown belgeyi karanlık temalı PDF'e çevirir

2. Dış bağımlılıklar — ne kullanıyor, ne için

Kit her şeyi kendi yapmaz; iki dış parçayı **çağırır**.

Addy Osmani'nin **agent-skills** paketi

25 skill içeren, İngilizce, süreç odaklı bir kütüphane. Kit **beşini** çağırır:

Skill	Kitin hangi adımında	Ne yapıyor
<code>interview-me</code>	Adım 3 (PRD görüşmesi)	Tek tek soru sorarak asıl isteneni çıkarır. Varsayım yapmadan, %95 netlik oluşana kadar sorar
<code>frontend-ui-engineering</code>	Arayüz kodu yazılırken	Bileşen kurgusu, durum yönetimi (state management) ve "Avoid the AI Aesthetic" bölümü — yapay zekânın tipik görsel varsayılanlarını (mor gradient, aşırı yuvarlak köşe, tek tip kart ızgarası) reddeder
<code>test-driven-development</code>	Özellik geliştirilirken	Önce başarısız test, sonra kod (RED → GREEN → REFACTOR)
<code>security-and-hardening</code>	Güvenlik denetiminde	Girdi doğrulama, oturum yönetimi, üçüncü parti entegrasyon riskleri
<code>code-review-and-quality</code>	Birleştirme (merge) öncesi	Doğruluk, okunabilirlik, mimari, güvenlik, performans — beş eksenli inceleme

⚠ **Çakışma olursa kitin kendi standardı üstündür.** Sebebi: kitin kuralları Türkçe ve bu stack'in token sistemine bağlı.

🚫 **Addy'nin paketinde SEO ile ilgili hiçbir şey yoktur** (ölçüldü: `seo`, `sitemap`, `canonical`, `schema.org` → sıfır sonuç). SEO tamamen kite aittir.

`chrome-devtools` MCP

Tarayıcıyı fiilen sürer: sayfayı açar, tıklar, ekran görüntüsü alır, konsol ve ağ hatalarını okur. Kitin "*kod okuyup çalışması lazım demek yeterli değil, kanıt getir*" kuralı ancak bununla geçilebilir.

Kullanılmayanlar

gstack ve **superpowers** kurulu **değildir**. Katkıları kural olarak kite alındı, paketleri yedeğe kaldırıldı. Sebep: dördü açıkken ~99 skill açıklaması her oturuma yüklenir (seçim gürültüsü), ve superpowers'ın "*insana sorma, devam et*" kuralı kitin "*plan sun, onayımı bekle*" kapısıyla çelişir.

3. Akış — `/yeni-proje` sekiz adımda ne yapar

Adım 0 — Bağımlılıklar

Addy'nin paketi ve chrome-devtools MCP kurulu mu bakar, **yalnızca eksik olanı** kurar. 🚫 Bu adım hiçbir skill'i çalıştırmaz, sadece kurar.

Adım 1 — Proje tipi ve stack

- Bu proje kimin için? `kendi projem` / `kurum projesi` — sonraki her şeyi bu belirler

- Web mi, mobil (Expo) mi, ikisi mi
- **Backend kurgusu:** Next.js tek başına mı, Next + NestJS mi
- Klasör boşsa devam eder; doluyorsa **ne olduğuna bakar** ve gerekirse durup sorar

Adım 2 — Kit dosyalarını yerleştir

19 standart (`docs/standards/00–18`), `CLAUDE.md` (ajan kuralları), `CALISMA–KILAVUZU.md` (kullanıcı kılavuzu), VS Code eklenti önerileri.

Adım 3 — PRD (en kritik adım)

`interview-me` ile **tek tek** soru. Netleşmeden geçilmez: kim kullanacak · hangi problem · **kapsam dışı ne** · roller · iş kuralları · hata durumları · **aynı anda kaç kişi ve ne zaman** · arka planda iş var mı.

★ **Değer sorusu:** her özellik için "yapılmalı mı" sorulur — "Bu ekran hangi problemi *çözüyor*? Olmasaydı kullanıcı ne yapardı?" Kurum projesinde ayrıca kıymetli: analiz birimi çoğu zaman **çözümü** yazar, **problemi** değil.

Görüşmenin sonunda tasarım yönü ve SEO kapsamı sorulur (ürün önce, görünüm sonra).

Adım 4 — Yol haritası ve ilk kararlar

Adımlar bağımlılık sırasına göre. ADR'ler (Architecture Decision Record — mimari karar kaydı) yazılır.

Kapanışta roadmap altı gözle denetlenir: ürün · risk · geri alınabilirlik · **dış bağımlılık** · mühendislik · kullanım.

Adım 5 — İskeleti kur

Framework, TypeScript strict, lint + format, git, `.env.example`.

Adım 6 — Yayın (Adım 1'deki cevaba göre ikiye ayrılır)

- **6a Kendi projem:** GitHub + hosting + veritabanı + CI + sağlık ucu (health endpoint) + **bölge eşleşmesi** + arama motoruna tanıtma
- **6b Kurum projesi:** deploy edilmez, DevOps'un çalıştıracağı **teslim paketi** üretilir (Dockerfile, docker-compose, `.env.example`, README) ve **kendi makinende doğrulanır**

Adım 7 — Son kontrol

On dört maddelik liste: şablonlar dolduruldu mu, tasarım ADR'si yazıldı mı, ekran görüntüsü alınıp **bakıldı** mı, SEO kuruldu mu, bölgeler eşleşti mi.

4. On dokuz standart

#	Dosya	Kapsam
00	Stack	Teknoloji seçimleri — başlangıç noktası, dondurulmuş liste değil . Her projede yeniden ölçülür
01	Mimari	Katman sırası: UI → API → Servis → Repository → Veritabanı. Katman atlanmaz
02	Kod standartları	Adlandırma, yorum, hata yönetimi, ESLint/Prettier çıkışması
03	API	Sözleşme (contract), sürümleme, hata biçimi, sayfalama (pagination)
04	Veritabanı	Tablo, ilişki, index, migration (şema göçü), yumuşak silme (soft delete)
05	Kimlik/Güvenlik	Oturum, token ömürleri, tokenVersion ile iptal, yetkilendirme
06	Test	Piramit: çok unit, orta entegrasyon, az uçtan uca (E2E)
07	Arayüz/Tasarım	Token'lar, karanlık tema, responsive, AI slop yasakları , hareket (motion), erişilebilirlik (a11y)
08	Git	Dal (branch), commit biçimi, PR, geri alma (rollback)
09	CI/CD	Paketleme, otomatik test, Lighthouse ve axe kapıları
10	Definition of Done	"Bitti" ne demek — işaretlenmeden iş bitmez
11	Ajanla çalışma	Bağlam yönetimi, belirsizlikte davranış, üçüncü başarısız düzeltmede dur
12	Çalıştırma/ Ölçekleme	İzleme, log, bölge eşleşmesi , ani yük (spike traffic)
13	Ortamlar	Local · preview · canlı; preview noindex
14	Gizlilik/KVKK	Hesap silme, veri hakları, denetim kaydı (audit log)
15	Oturum devri	Sonraki oturumun bilmesi gereken her şey dosyaya yazılır
16	Proje kurulumu	Kurulum protokolü
17	Mobil	React Native + Expo, mağaza yayını
18	SEO	Render stratejisi, URL, meta, JSON-LD, site haritası, Search Console

5. Ajan kapıları — **CLAUDE.md**

Kitin en belirleyici parçası. Ajanın **neyi yapamayacağını** söyler:

Kapı	Kural
1	Kod okuyup "çalışması lazım" demek yetmez — kanıt getir (tarayıcıda tıkla)
2	Plan sun, onay bekle — kod yazmadan önce, her zaman
3	Bir ADR'ye aykırı kod yazılmaz; karar değişecekse önce yeni ADR

Ve iş bölümü kuralı: 🚫 **Ajanın yapabildiği hiçbir iş kullanıcıya yaptırılmaz.** Kullanıcının zamanı yalnızca ajanın *yapamadığı* işler için harcanır (hesap açma, ödeme, kurumdan yetki alma).

6. Bu kit nasıl gelişiyor

Kit, gerçek projelerde yapılan hatalardan büyüyor. Döngü:

Projede bir hata yapılır veya daha iyi bir yol bulunur

↓

/kit-senkron çalıştırılır

↓

Fark üç kutudan birine konur:

- Kalıcı kural → kite yazılır
- Projeye özel → projede kalır
- Kitten gelen yeni → projeye getirilir

↓

Sürüm artırılır, GitHub'a push edilir

↓

Kullanan herkes /plugin update ile çeker

🚫 **Bir kural projeye özel hâle geliyorsa o kural yanlış yazılmıştır.** Kural düzeltilir, projeye göre dallandırılmaz. Framework'ü çürüten şey dallanmadır: üç proje sonra elinde birbirinden sapmış üç kopya olur.

Bu belge **proje-kiti** v1.74.0 için üretilmiştir. Kit değişikçe güncellenir.